

SDLC - Copilot

Lab 02.

Step 1: Generate a Class Using Copilot

The screenshot displays the Visual Studio Code interface with the SDLC project open. The Explorer pane on the left shows the project structure, including the 'SDLC/CopilotLabs' directory. The main editor window shows the 'lab02.py' file, which contains a Python class named 'StudentManagement'. The class has a constructor, four methods (add_student, display_student, update_student, delete_student), and a main function. The Chat pane on the right shows the Copilot chat interface, which has generated the code for the 'StudentManagement' class. The chat messages indicate that Copilot is generating the code and will validate the syntax. The terminal pane at the bottom shows the output of the Python script, which includes the class definition and the execution of the main function.

```
# Step 1: Generate a class using Copilot
1 class StudentManagement:
2     def __init__(self, student_id, student_name, department):
3         self.student_id = student_id
4         self.student_name = student_name
5         self.department = department
6
7     def add_student(self):
8         """Returns a confirmation message for adding a student."""
9         return (
10             f"Student added: ID={self.student_id}, "
11             f"Name={self.student_name}, Department={self.department}"
12         )
13
14     def display_student(self):
15         """Returns student details."""
16         return {
17             "student_id": self.student_id,
18             "student_name": self.student_name,
19             "department": self.department,
20         }
21
22     def update_student(self, student_name=None, department=None):
23         """Updates student name and/or department."""
24         if student_name is not None:
25             self.student_name = student_name
26         if department is not None:
27             self.department = department
28         return "Student record updated successfully."
29
30     def delete_student(self):
31         """Clears student details to represent deletion."""
32         self.student_id = None
33         self.student_name = None
34         self.department = None
35         return "Student record deleted successfully."
36
37
38 if __name__ == "__main__":
39     student = StudentManagement(101, "Aarav", "Computer Science")
40     print(student.add_student())
41     print(student.display_student())
42
43     print(student.update_student(student_name="Aarav Sharma", department="AI & DS"))
44     print(student.display_student())
45
46     print(student.delete_student())
47     print(student.display_student())
48
```

Chat messages:

Python CLASS STUDENTMANAGEMENT...

Create a Python class named StudentManagement with:

- student_id
- student_name
- department

Functions:

- add_student()
- display_student()
- update_student()
- delete_student()

Use constructor and proper OOP practices.

Optimized tool selection

I'll open the current lab02.py file and implement a clean StudentManagement class with a constructor and the four required methods, then I'll quickly validate for syntax issues.

I'm gathering the file's current contents first so I can apply the smallest targeted edit and preserve any existing work.

Read • lab02.py

1 file changed +47 -1 Keep Undo

+ lab02.py

Describe what to build

+ Auto

Local Default Approvals

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + Python 3.14.2

```
/usr/local/bin/python3 /Users/su28546756/Desktop/GENAI_WE_PROG/SDLC/CopilotLabs/Day02/lab02.py
su28546756@FVFXHGBVQ6LR GENAI_WE_PROG % /usr/local/bin/python3 /Users/su28546756/Desktop/GENAI_WE_PROG/SDLC/CopilotLabs/Day02/lab02.py
Student added: ID=101, Name=Aarav, Department=Computer Science
{'student_id': 101, 'student_name': 'Aarav', 'department': 'Computer Science'}
Student record updated successfully.
{'student_id': 101, 'student_name': 'Aarav Sharma', 'department': 'AI & DS'}
Student record deleted successfully.
{'student_id': None, 'student_name': None, 'department': None}
```

Step 2: Analyze the Code

Analyze this Python code and explain its structure, strengths, and possible improvements.

I'll inspect the latest contents of the file first, then I'll give you a focused review of the structure, what is working well, and concrete improvements with examples.

Read [lab02.py](#)

Structure

The code in [lab02.py](#) is organized into two clear parts:

1. StudentManagement class definition at [lab02.py:1](#)
2. A runnable demo block under main at [lab02.py:35](#)

Method layout is straightforward:

1. Constructor initializes state at [lab02.py:2](#)
2. Add operation returns a confirmation string at [lab02.py:7](#)
3. Display operation returns a dict snapshot at [lab02.py:14](#)
4. Update operation supports partial updates at [lab02.py:22](#)
5. Delete operation resets fields to None at [lab02.py:30](#)

Strengths

1. Good OOP baseline: state is encapsulated in one class with behavior grouped around that state.
2. Constructor usage is correct and clean for initialization.
3. Methods have single, understandable responsibilities.
4. Partial update design is practical: update only what is passed.
5. Docstrings are present and improve readability.
6. Main guard pattern is correctly used, so import side effects are avoided.

Possible Improvements

[+ lab02.py](#)

Step 3: Code Review

< PYTHON CLASS STUDENTMANAGEMENT DESIGN

7. Add minimal unit tests with `pytest` for core flows.

Small Best-Practice Refactor Sketch

```
from dataclasses import dataclass, asdict

@dataclass
class Student:
    student_id: int
    student_name: str
    department: str

    def __post_init__(self) -> None:
        if self.student_id <= 0:
            raise ValueError("student_id must be positive")
        if not self.student_name.strip():
            raise ValueError("student_name cannot be empty")
        if not self.department.strip():
            raise ValueError("department cannot be empty")

    def display_student(self) -> dict:
        return asdict(self)

    def update_student(self, student_name: str | None = None, department: str | None = None) -> None:
        if student_name is not None:
            if not student_name.strip():
                raise ValueError("student_name cannot be empty")
            self.student_name = student_name
        if department is not None:
            if not department.strip():
                raise ValueError("department cannot be empty")
            self.department = department
```

If you want, I can apply these improvements directly to [lab02.py](#) and keep method names compatible with your assignment requirements.

[GPT-5.3-Codex • 2.5 credits](#)

[+ lab02.py](#)

Describe what to build

[+ Agent](#) [Auto](#)

Local | Default Approvals

Step 4: Try Different Models

Priority 1 — Correctness

#	Issue	Location	Fix
1	<code>add_student()</code> just formats a string; it never persists anything	• lab02.py:7	Rename to <code>get_info()</code> or redesign to use a collection
2	<code>delete_student()</code> nulls fields but object still lives	• lab02.py:30	Use a manager dict and actually <code>del students[id]</code>

Priority 2 — Robustness

#	Issue	Location	Fix
3	No validation of inputs in constructor or <code>update_student</code>	• lab02.py:2 , • lab02.py:22	Raise <code>ValueError</code> for empty name, non-positive ID
4	<code>update_student()</code> silently succeeds even after <code>delete_student()</code> sets fields to <code>None</code>	• lab02.py:22	Guard against deleted state

Priority 3 — Code Quality

#	Issue	Fix
5	Tabs used instead of 4 spaces	Run <code>autoflake --in-place lab02.py</code> or configure editor to use spaces
6	No type hints on any method	Add <code>student_id: int</code> , <code>student_name: str</code> , <code>-> dict</code> , <code>-> None</code> , etc.
7	Inconsistent returns (some <code>str</code> , one <code>dict</code>)	Mutators return <code>None</code> ; raise on error. Readers return structured data
8	No <code>__repr__</code> / <code>__str__</code>	Add so <code>print(student)</code> gives useful output instead of <code><StudentManagement object></code>

Priority 4 — Design

#	Issue	Fix
9	Class name implies collection management but only holds one student	Either rename to <code>Student</code> or add an internal <code>dict</code> registry

lab02.py

Describe what to build

Agent

Claude Sonnet 4.6

High 1M

LocalDefault Approvals

Ln 48, Col 1 Spaces: 4 UTF-8 LF Python Python 3

Step 5: Slash Commands

/explain

Key Improvements:

1. Type Hints — Added annotations for parameters and return types for clarity and IDE support.

2. Input Validation — Constructor and `update_student()` now validate types and non-empty strings.

3. Deleted State Guard — Introduced `_is_deleted` flag; methods raise `RuntimeError` when called on a deleted record (better than silently allowing invalid state).

4. `__repr__` Method — Makes `print(student)` output useful for debugging.

5. PEP 8 Spacing — Replaced tabs with 4 spaces throughout.

6. Enhanced Docstrings — More descriptive, include validation behavior.

Usage Example:

```
if __name__ == "__main__":
    student = StudentManagement(1, "Alice", "Computer Science")
    print(student)  # StudentManagement(id=1, name=Alice, dept=Computer Science)
```

/fix

Restore Checkpoint 1

lab02.py:1-35

/fix

Problem Identified

The bug is in the `if __name__ == "__main__":` block at the end. After calling `student.delete_student()`, the code tries to call `student.display_student()`, which raises a `RuntimeError` because the student record is marked as deleted.

This is a logic error—the code attempts an invalid operation without handling it.

Fixed Code

```
class StudentManagement:
    def __init__(self, student_id: int, student_name: str, department: str) -> None:
        """Initialize a student record with validation."""
        if not isinstance(student_id, int) or student_id <= 0:
            raise ValueError("student_id must be a positive integer")
        if not isinstance(student_name, str) or not student_name.strip():
            raise ValueError("student_name must be a non-empty string")
        if not isinstance(department, str) or not department.strip():
            raise ValueError("department must be a non-empty string")

        self.student_id: int = student_id
        self.student_name: str = student_name
        self.department: str = department
        self._is_deleted: bool = False

    def __repr__(self) -> str:
        """Return a developer-friendly string representation."""
        return (
            f"StudentManagement(id={self.student_id}, "
            f"name={self.student_name!r}, dept={self.department!r})"
        )
```

lab02.py:1-35

Describe what to build

/tests

lab02.py:1-35

/tests

Plan

1. Import `StudentManagement` from `lab02.py` using absolute import (namespace package, no `__init__.py`)
2. Use `pytest` framework for unit tests
3. Cover all methods: `__init__`, `add_student()`, `display_student()`, `update_student()`, `delete_student()`, `__repr__()`
4. Test happy paths, validation errors, and edge cases
5. Test deleted state guards on all mutating/reading methods
6. Organize tests by method with clear naming

```
import pytest
from SDLC.CopilotLabs.Day02.lab02 import StudentManagement

class TestStudentManagementInit:
    """Test StudentManagement constructor and validation."""

    def test_init_valid_student(self) -> None:
        """Test successful initialization with valid inputs."""
        student = StudentManagement(1, "Alice", "Computer Science")
        assert student.student_id == 1
        assert student.student_name == "Alice"
        assert student.department == "Computer Science"
        assert student._is_deleted is False

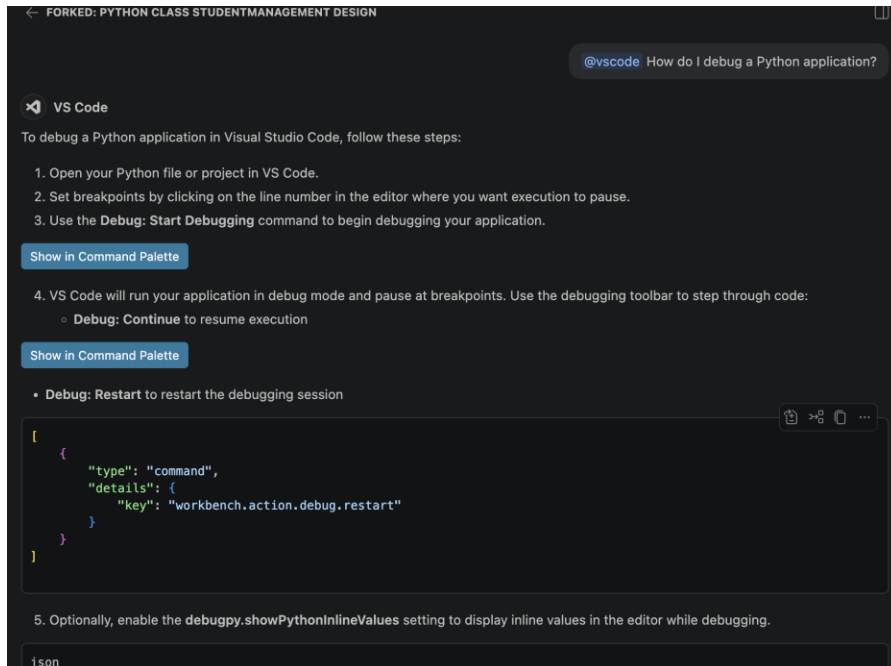
    def test_init_invalid_student_id_zero(self) -> None:
        """Test that student_id cannot be zero."""
        with pytest.raises(ValueError, match="student_id must be a positive integer"):
            StudentManagement(0, "Bob", "Physics")

    def test_init_invalid_student_id_negative(self) -> None:
        """Test that student_id cannot be negative."""
```

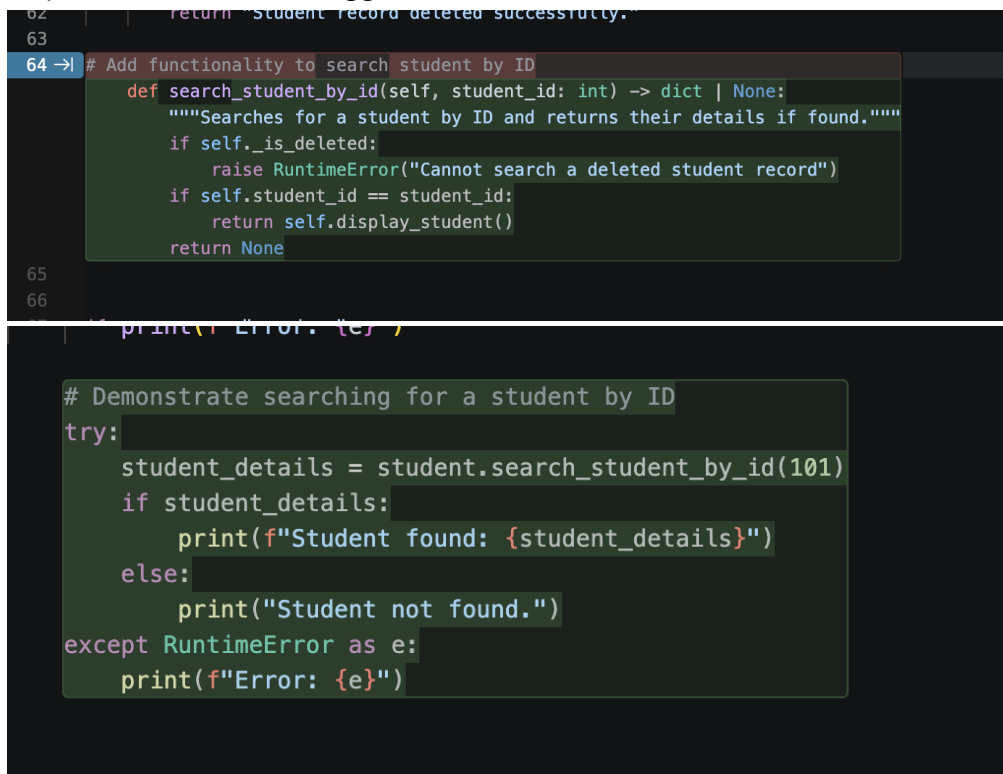
lab02.py:1-35

Step 6: Use @ Mentions

@vscode



Step 7: Comment-Based Suggestions



Step 8: Convert Python to Java

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

/**
 * StudentManagement class represents a student record with validation.
 * Provides CRUD operations on a single student entity.
 */
public class StudentManagement {
    private int studentId;
    private String studentName;
    private String department;
    private boolean isDeleted;

    /**
     * Constructor to initialize a student record with validation.
     *
     * @param studentId must be a positive integer
     * @param studentName must be a non-empty string
     * @param department must be a non-empty string
     * @throws IllegalArgumentException if any parameter is invalid
     */
    public StudentManagement(int studentId, String studentName, String department) {
        if (studentId <= 0) {
            throw new IllegalArgumentException("student_id must be a positive integer");
        }
        if (studentName == null || studentName.trim().isEmpty()) {
            throw new IllegalArgumentException("student_name must be a non-empty string");
        }
        if (department == null || department.trim().isEmpty()) {
            throw new IllegalArgumentException("department must be a non-empty string");
        }

        this.studentId = studentId;
        this.studentName = studentName;
        this.department = department;
        this.isDeleted = false;
    }

    /**
     * Returns a developer-friendly string representation of the student.
     *
     * @return formatted string representation
     */
}
```

Step 9: Non-Coding Question

Start Over

Recipe for Palak paneer

Searched for Palak paneer recipe

Palak Paneer Recipe

Ingredients:

For the palak (spinach) base:

- 500g fresh spinach (or 250g frozen)
- 1 onion, chopped
- 4-5 garlic cloves
- 1-inch ginger, chopped
- 2-3 green chillies
- 1/4 cup cream or yogurt
- 2 tbsp ghee or oil
- Salt and pepper to taste
- 1/2 tsp cumin seeds

For paneer:

- 300g paneer, cut into cubes
- Oil for frying

Instructions:

1. **Blanch spinach:** Boil water, add spinach and blanch for 2 minutes. Drain and cool, then grind into a smooth paste with

Step 10: Context Files

3. Design Improvements Summary

Issue	Current	Recommended
Single vs Multiple	Single student per class	Separate Student model + StudentManagementSystem for collection
Validation	Only in lab02.py	Implement in base Student class
Type Hints	Missing in student.py	Add throughout
Naming	Inconsistent	Use consistent: <code>student_id</code> , <code>name</code> , <code>department</code>
Deletion	Soft delete flag only	Soft delete with <code>is_active</code> boolean
Persistence	None	Add file/database support (future)
Error Handling	Present in lab02.py	Standardize across both

4. Additional Enhancements to Consider

- **Factory pattern:** Create a `StudentFactory` for object creation
- **Search capabilities:** Filter by department, search by name
- **Data persistence:** Add JSON/CSV export/import methods
- **Logging:** Track add/update/delete operations
- **Batch operations:** Add multiple students at once

This restructuring provides **separation of concerns**, **reusability**, and **scalability** for managing student records.